

# **FRAMEWORK PER L'AUTOMAZIONE DEI TEST**

**Michele Marchesi**

**marchesi@unica.it**

**Corso di Ingegneria del Software  
A.A. 2025/26**



# Introduzione

---

- In queste slide si presenta l'intero approccio per effettuare test automatici, specificamente in ambiente Python
- All'inizio, è presentato l'approccio "Behavior-Driven Development (BDD) e il linguaggio Gherkin per descrivere il comportamento di un sistema
- Segue la descrizione dei framework per i test automatici



# Behavior-Driven Development

- Idea fondamentale: parlare di ciò che il sistema deve fare in termini di valore e risultati visibili.
- Domande tipiche: *“Dato un certo contesto, cosa succede quando l’utente esegue un’azione?”*
- Strumenti tipici: scenari condivisi passo-passo che possono essere automatizzati
- Il BDD parte dai comportamenti attesi in fase di specifica, e usa esempi eseguibili e linguaggio condiviso.
-  Un modo per esprimere i requisiti funzionali.

# Il linguaggio Gherkin



- Gherkin (significa: *cetriolo aromatizzato*) è il linguaggio di Cucumber, il framework storico del BDD per Ruby e poi Java.
- È usato per descrivere i test di accettazione o funzionali.
- Gherkin descrive flussi utente, regole di business ed esiti attesi in modo leggibile.



# Il flusso tipico del BDD

- Nel mondo Python, una combinazione comune è: file Gherkin \*.feature + behave + definizioni in Python dei passi + Selenium/Playwright/API client:



# Feature e Scenario

- Sono le due strutture di base di Gherkin
- Feature descrive la funzionalità o l'area di business
- Scenario descrive un caso specifico da verificare
- Una Feature può contenere più scenari
- Spesso sotto Feature si aggiunge una breve descrizione del **valore atteso**

Feature: Gestione login

Per proteggere l'accesso al sistema

Come utente registrato

Voglio autenticarmi con email e password

Scenario: Login con credenziali corrette

Given un utente registrato

When inserisce credenziali valide

Then accede alla dashboard

# Given / When / Then

- Given: definisce il contesto iniziale o le precondizioni.
- When: descrive l'azione eseguita o l'evento che accade.
- Then: descrive il risultato atteso, osservabile dall'esterno.
- I passi dovrebbero descrivere comportamenti osservabili, non dettagli interni del codice

Scenario: Ricarica del portafoglio

Given un utente autenticato con saldo 10 euro

When ricarica 20 euro

Then il saldo diventa 30 euro



# Background

- Background contiene precondizioni comuni a tutti gli scenari della stessa Feature
- Riduce la duplicazione quando più scenari partono dallo stesso contesto
- Va usato con misura: troppo Background rende i file meno leggibili

**Feature:** Gestione portafoglio

**Background:**

Given un utente autenticato

And il suo portafoglio è attivo

**Scenario:** Visualizzare il saldo

When apre la pagina del portafoglio

Then vede il saldo corrente



# Data

- Scenario Outline permette di riusare lo stesso scenario con più insiemi di dati
- I segnaposto si scrivono tra parentesi angolari, per esempio <email>
- Examples contiene la tabella dei valori; ogni riga produce un'esecuzione dello scenario

Scenario Outline: Login con diversi input

Given un utente registrato

When inserisce email "<email>" e password "<password>"

Then l'esito è "<esito>"

Examples:

| email        | password | esito    |  |
|--------------|----------|----------|--|
| demo@test.it | 1234     | fallito  |  |
| demo@test.it | segreta  | riuscito |  |



# Data Tables

- Servono per passare dati strutturati:

Scenario: Creazione utenti

Given esistono i seguenti utenti:

|       |               |  |
|-------|---------------|--|
| nome  | email         |  |
| Mario | mario@test.it |  |
| Luca  | luca@test.it  |  |

Then gli utenti sono disponibili



# Doc Strings

- Servono per passare testi, anche su più righe:

```
Scenario: Inserimento descrizione
  When l'utente inserisce il testo:
    """
    Riga uno
    Riga due
    """
  Then la descrizione viene salvata
```



# Tag

- Classificano feature o scenari:
  - @smoke: “Smoke test” (test funzionale veloce e superficiale di verifica iniziale)
  - @regression: test di regressione
  - @api: lo scenario riguarda test di API: chiamate HTTP, endpoint REST, request/response, ecc.
  - @ui: lo scenario riguarda la GUI (Web browser, form, pagine, ecc.)
  - @slow. indica che il test è lento

```
@smoke @login
```

```
Scenario: Login corretto
```

```
Given un utente registrato
```

```
When inserisce credenziali valide
```

```
Then accede alla dashboard
```



# Struttura tipica di un file .feature

@wallet

Feature: Ricarica del portafoglio

Background:

Given un utente autenticato

@smoke

Scenario: Ricarica valida

Given il saldo è 10 euro

When ricarica 20 euro

Then il saldo è 30 euro

Scenario Outline: Ricariche non valide

When ricarica "<importo>" euro

Then la ricarica viene rifiutata

Examples:

|  |         |  |
|--|---------|--|
|  | importo |  |
|  | 0       |  |
|  | -5      |  |

# BDD e Casi d'Uso

- Hanno legami, ma non sono la stessa cosa
- I casi d'uso descrivono:
  - attori
  - obiettivo
  - flusso principale
  - flussi alternativi
  - precondizioni e postcondizioni
- Il BDD si concentra su:
  - esempi concreti, regole di business
  - scenari verificabili, comportamento osservabile
- Da un CdU si possono derivare più scenari BDD, e relativi TA



# Il framework XUnit

- xUnit è una famiglia di framework per test unitari.
- Sviluppato originariamente da Kent Beck per il linguaggio Smalltalk (SUnit)
- Facilita la scrittura e l'esecuzione dei test unitari, permette la ripetibilità e l'automazione dei test
- Esistono versioni per Java (JUnit), C++, C#, VB, Ada, C, .Net, Objective-C, Oracle, Perl, Python, Ruby, VOC, ... Vedi: [https://en.wikipedia.org/wiki/List\\_of\\_unit\\_testing\\_frameworks](https://en.wikipedia.org/wiki/List_of_unit_testing_frameworks)
- Quasi tutti i test framework di Python sono Xunit-like
- Un **Test runner** è un programma eseguibile che esegue test implementati utilizzando un framework xUnit e riporta i risultati del test



# X-Unit

- 
- X-Unit si basa su quattro pattern:
  - **Fixture**: insieme di variabili od oggetti, inizializzati a valori opportuni, usati ripetutamente come dati di input per i test.
    - Ogni volta che un test è “lanciato”, la sua Fixture è reinizializzata.
    - **setUp()** metodo che crea o reinizializza la Fixture
    - **tearDown()** rilascia ogni risorsa usata per il test



# X-Unit

- Esempio in Python:

```
import unittest
```

```
class TestCalcolatrice(unittest.TestCase):
```

```
    def setUp(self):
```

```
        # Inizializzazione della Fixture
```

```
        self.calc = Calcolatrice()
```

```
    def tearDown(self):
```

```
        del self.calc # Pulizia della Fixture
```



# X-Unit

- **Test-Case:** è un singolo test, o una serie di test atomici, definito su una Fixture e implementata in codice, in modo tale da essere ripetibile e trasmissibile ad altri sviluppatori

- Esempio in Python:

```
def test_somma_due_numeri(self):  
    risultato = self.calc.somma(2, 3)  
    self.assertEqual(risultato, 5)
```



# X-Unit

- **Check:** un test case stimola una Fixture e controlla i risultati attesi. Si usano funzioni di controllo (Check) capaci di testare condizioni Booleane e riportare i risultati e lo stato del sistema in caso di fallimenti.
- Ad es.:
  - **assert**(*boolean expression*)
  - **equals**(*expression1, expression2*)
  - **equalsDouble**(*num expression1, num expression2*)



# X-Unit

- Esempio in Python:

```
import unittest
# funzioni da testare
def is_pari(n):
    return n % 2 == 0
```

```
def somma(a, b):
    return a + b
```

```
def radice_quadrata(x):
    if x < 0:
        raise ValueError("Radice di numero neg-
ativo non definita")
    return x ** 0.5
```

# X-Unit

```
class TestFunzioniMatematiche(unittest.TestCase):  
    def test_is_pari(self):  
        self.assertTrue(is_pari(4))  
  
    def test_somma(self):  
        self.assertEqual(somma(2, 3), 5)  
  
    def test_radiceQuadrata(self):  
        self.assertAlmostEqual(radice_quadrata(2)**2, 2.0,  
places=5)  
  
    def test_radiceQuadrata_2(self):  
        with self.assertRaises(ValueError):  
            radice_quadrata(-9)
```



# X-Unit

- **Test Suite:** i Test Case sono raggruppati e organizzati in Test Suite, insiemi di test case che possono essere lanciati automaticamente e ripetutamente.
  - Quando una Test Suite è “lanciata” tutti i suoi Test Case sono eseguiti e tutti gli **error** e i **failure** sono mostrati.
  - In Python:

```
if __name__ == '__main__':  
    unittest.main()
```



# X-Unit

- Tipi di Errori nei test:
  - **Error**: errore di esecuzione che porta a un'eccezione non gestita o a un crash di sistema. Nei linguaggi interpretati è “catturato” dal test runner, segnalato e i test possono continuare!
  - **Failure**: errore dei dati: un Check non è superato, ma il codice non abortisce.
  - **ATTENZIONE**: *Non corrispondono alle definizioni IEEE date all'inizio delle slide sul testing!*



# X-Unit

- L'esecuzione di un singolo test unitario procede come segue:

```
setup(); /* First, we should prepare our 'world'
to make an isolated environment for testing */
...
/* Body of test - Here we make all the tests */
...
teardown(); /* At the end, whether we succeed or
fail, we should clean up our 'world' to
not disturb other tests or code */
```



# J-unit

- E' un framework di testing open source per **Java** scritto da Eric Gamma e Kent Beck
- Si usa per white box testing
- Disponibile su [www.junit.org](http://www.junit.org)
- Permette di scrivere codice più velocemente, aumentando la qualità
  - Diminuisce i tempi di debug
  - Aumenta la confidenza tra gli sviluppatori
  - Permette il refactoring



Eric e Kent a  
XP2000, Cagliari



# Mock object

- Il ***mock object*** è un concetto fondamentale legato ai test automatici
- È un oggetto fittizio usato nei test per simulare il comportamento di un oggetto necessario al test, ma non ancora implementato o problematico da usare
- Serve a isolare l'unità di codice da testare, o a evitare chiamate reali a database, API, file system o servizi esterni
- L'obiettivo è rendere i test più rapidi, prevedibili e isolati



# Framework di testing per Python

---

- Di seguito, si riportano i principali framework per eseguire test automatici in Python
- Il settore è in continua evoluzione, per cui tutti i dati vanno rivalutati in continuazione
- Framework citati:
  - Test Unitari: unittest, pytest, nose2
  - Test di Accettazione: behave, Robot Framework
  - Test di Web app: Selenium, Playwright



# unittest

- E' il framework di testing per **Python** originariamente ispirato da X-Unit: si usa per white box testing
- Disponibile come package standard Python
- Supporta assertion, setup/teardown, suite e discovery automatica dei test.
- È una scelta solida in contesti enterprise o dove si preferisce una base standard.
- Documentazione:  
*<https://docs.python.org/3/library/unittest.html>*



# unittest: quando usarlo

---

- Usarlo per: librerie interne, codice legacy, ambienti con politiche conservative.
- Punti forti: standard, molto diffuso, facilissimo da trovare già disponibile.
- Limiti: più verboso di pytest e meno espressivo per fixture e parametrizzazione.
- Best practice: mantenere i TestCase sintetici, usare mock-object solo sui confini esterni.



# unittest

```
import unittest

class TestStringMethods(unittest.TestCase): # sottoclasse di TestCase

    def test_upper(self):
        self.assertEqual('foo'.upper(), 'F00')
    def test_isupper(self):
        self.assertTrue('F00'.isupper())
        self.assertFalse('Foo'.isupper())
    def test_split(self):
        s = 'hello world'
        self.assertEqual(s.split(), ['hello', 'world'])
        # check that s.split fails when the separator is not a string
        with self.assertRaises(TypeError):
            s.split(2)
```



# unittest

```
1 import unittest
2
3 class Calculator:
4     def add(self, a, b):
5         return a + b
6
7 class TestCalculator(unittest.TestCase):
8     def setUp(self):
9         self.calc = Calculator()
10
11     def test_add(self):
12         self.assertEqual(self.calc.add(3, 4), 7)
```



# pytest

- È il framework oggi più usato per test Python
- Permette test semplici come funzioni normali, senza classi obbligatorie.
- Offre asserzioni leggibili, fixture riusabili, parametrizzazione e molti plugin dedicati, ad es:
  - pytest-cov per coverage
  - pytest-xdist per eseguire i test in parallelo
  - pytest-mock per usare meglio i mock object
  - pytest-django per progetti Django
- Le fixture possono essere condivise tramite `conftest.py`.



# pytest

- Scala bene da unit test a test funzionali e API test.
- Spesso è la scelta predefinita per nuovi progetti Python.
- Punti forti: leggibilità, bassa verbosità, ottimo supporto a plugin e integrazione continua.
- Fixture: costruiscono un contesto testabile affidabile e riutilizzabile.
- Ideale anche per Flask, Django, FastAPI e test API.
- Documentazione: (<https://docs.pytest.org/>)



# pytest

```
import pytest

@pytest.fixture
def numbers():
    return [1, 2, 3]

@pytest.mark.parametrize("value,expected", [(2, True), (5, False)])
def test_contains(numbers, value, expected):
    assert ((value in numbers) == expected)

def test_split():
    s = 'hello world'
    assert s.split() == ['hello', 'world']

    # check che s.split fallisca se il separatore non è una stringa
    with pytest.raises(TypeError):
        s.split(2)
```



# nose2

- È basato su unittest, quindi eredita buona parte dei concetti di tale framework standard.
- Aggiunge un sistema a plugin e un runner più estendibile.
- Può essere utile in progetti che partono già da unittest e vogliono estensioni progressive.
- Oggi però, in molti nuovi progetti, è spesso preferito pytest .



# behave

- È orientato al BDD: i test vengono descritti in linguaggio naturale strutturato: Given / When / Then.
- È l'implementazione Python del framework BDD "Cucumber" per Ruby, Java e Javascript.
- Aiuta la collaborazione tra sviluppatori, QA e altri stakeholder
- Dietro ogni step c'è codice Python che implementa il comportamento atteso.
- Si usa per test di accettazione, test di regressione su scenari business, collaborazione con QA.
- Molto leggibile, forte allineamento coi requisiti
- Limiti: test di difficile manutenzione se gli step diventano troppo granulari.



# Da Gherkin a Python con behave

```
# Gherkin:
Scenario: Login corretto
    Given un utente registrato
    When inserisce credenziali valide
    Then accede alla dashboard

from behave import given, when, then

@given('un utente registrato')
def step_user(context):
    context.user = 'demo@example.com'

@when('inserisce credenziali valide')
def step_login(context):
    pass # qui chiameresti UI o API

@then('accede alla dashboard')
def step_result(context):
    assert True
```

# Robot

- Framework estensibile, basato su keyword e usato per l'automazione dei TA End-to-End (E2E).
- I test sono leggibili anche da figure che non sono strettamente analisti/programmatori.
- Può integrare librerie Python e automazione di interazione tramite browser.
- È adatto quando la leggibilità dei casi di test è un obiettivo primario.
- La scrittura dei test automatici è poi fatta in Python dagli sviluppatori (ad es. con Selenium).



# Robot Framework

## Scenario:

```
*** Test Cases ***  
2 Login riuscito  
3     Open Browser      https://example.com      chrome  
4     Input Text        id=email      demo@example.com  
5     Click Button      id=login  
6     Page Should Contain Dashboard
```

## Codice Python:

```
from selenium import webdriver  
from selenium.webdriver.common.by import By  
  
driver = webdriver.Chrome()  
driver.get("https://example.com")  
driver.find_element(By.ID, "email").send_keys("demo@example.com")  
driver.find_element(By.ID, "login").click()  
assert "Dashboard" in driver.page_source  
driver.quit()
```



# Selenium

- Framework storico, per automatizzare l'interazione con i browser tramite WebDriver.
- Serve per test E2E dell'interfaccia utente.
- Può essere combinato con pytest, behave o Robot.
- Ecosistema ampio, adatto a molte integrazioni enterprise.
- Per stack già consolidati con WebDriver/Grid, Selenium resta una scelta eccellente.
- Sarà presentato nel laboratorio Python.



# Playwright

---

- Framework che offre automazione moderna multi-browser e, per Python, raccomanda l'uso con plugin pytest.
- Come Selenium, serve per test E2E della GUI.
- Ha un setup moderno, gestione dei browser, ed è facile da azionare.
- Per nuovi test Web Python, Playwright + pytest è spesso un'opzione molto efficace.



# Selenium vs. Playwright

## Selenium:

```
1 from selenium import webdriver
2
3 driver = webdriver.Chrome()
4 driver.get("https://example.com")
5 assert "Example" in driver.title
6 driver.quit()
```

## Playwright:

```
1 def test_homepage(page):
2     page.goto("https://example.com")
3     assert "Example" in page.title()
```



# Tabella comparativa

| Framework  | Livello | Punti forti                              | Quando usarlo                         |
|------------|---------|------------------------------------------|---------------------------------------|
| unittest   | Unit    | standard library, stabile, classico      | legacy o contesti conservativi        |
| pytest     | Unit    | fixture, plugin, leggibilità             | scelta predefinita per nuovi progetti |
| nose2      | Unit    | plugin sopra test unitari                | migrazioni o contesti già esistenti   |
| behave     | TA/BDD  | scenari business leggibili               | acceptance test guidati da requisiti  |
| Robot      | TA/E2E  | keyword-driven, leggibile                | QA cross-team e suite E2E             |
| Selenium   | UI/E2E  | WebDriver, ecosistema maturo             | stack enterprise browser automation   |
| Playwright | UI/E2E  | multi-browser moderno, ottimo con pytest | nuovi test web end-to-end             |

